Quantum Neural Networks and Quantum Physics-Informed Neural Networks

From Input Encoding to PDE Solvers

Morten Hjorth-Jensen

Department of Physics, University of Oslo

May 6, 2026

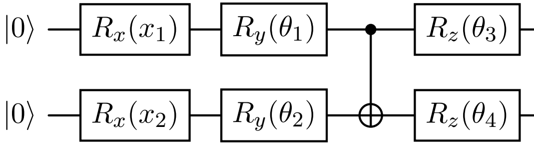
Outline

QNN as a Variational Quantum State

The central object of a **Quantum Neural Network (QNN)**:

$$|\psi(x,\theta)\rangle = U(\theta)\, U(x)\, |0\rangle^{\otimes n}$$

$$f(x,\theta) = \langle \psi(x,\theta)|\, O\, |\psi(x,\theta)\rangle$$



- $U(x)$: **feature map** — encodes classical input
- $U(\theta)$: **variational ansatz** — trainable
- O: **observable** — defines the output scalar

Parameter-shift rule

$$\frac{\partial f}{\partial \theta_j} = \frac{1}{2}\left[f(\theta_j + \tfrac{\pi}{2}) - f(\theta_j - \tfrac{\pi}{2})\right]$$

Exact gradient; 2 circuit evaluations per parameter.

Pauli rotation ansatz

$$U(\theta) = \prod_\ell e^{-i\theta_\ell P_\ell/2}, \qquad P_\ell \in \{I, X, Y, Z\}^{\otimes n}$$

QFI, TDVP, and Barren Plateaus: Quick Recap

Quantum Fisher Information (QFI):

$$F_{jk} = 4\,\mathrm{Re}(\langle \partial_j \psi | \partial_k \psi \rangle - \langle \partial_j \psi | \psi \rangle \langle \psi | \partial_k \psi \rangle)$$

Riemannian (Fubini–Study) metric on the variational manifold.

TDVP equations of motion:

$$\sum_k F_{jk}\,\dot{\theta}_k = h_j, \quad h_j = \mathrm{Im}\langle \partial_j \psi | H | \psi \rangle$$

Natural gradient:

$$\dot{\boldsymbol{\theta}} = -\mathbf{F}^{-1} \nabla_\theta \mathcal{L}$$

BCH and depth–expressivity trade-off

$$U^\dagger O\, U = O + i[H_\theta, O] + \frac{(i)^2}{2!}[H_\theta, [H_\theta, O]] + \cdots$$

Depth $\leftrightarrow$ hierarchy of correlations.

Barren plateaus

Deep random circuits form approximate unitary 2-designs:

$$\mathrm{Var}\left(\frac{\partial \mathcal{L}}{\partial \theta_j}\right) \sim \frac{1}{2^n} \to 0$$

Mitigation: problem-inspired *shallow*

From Discriminative QNNs to Physics-Informed QNNs

Discriminative QNN

- Input: labelled data pair (x_k, y_k)
- Loss: $\mathcal{L} = \sum_k (f(x_k;\theta) - y_k)^2$
- Learns a mapping from observations
- No physical constraint enforced

Physics-informed QNN (QPINN)

- Input: collocation points $x_k \in \Omega$
- Loss: PDE residual + boundary conditions

$$\mathcal{L} = \|\mathcal{D}[f_\theta]\|^2 + \lambda \|\mathcal{B}[f_\theta]\|^2$$

- Learns the *solution* of the PDE

Why quantum circuits for PDEs?

1. Potential exponential compression of solution space via entanglement
2. Natural encoding of wave-like solutions through rotation gates
3. Input-shift rule gives spatial derivatives without additional circuit depth
4. Connection to quantum simulation of differential operators
5. QFI and TDVP provide the optimal optimisation geometry (covered in detail in this lecture)

Why Encoding Matters

A variational quantum circuit takes classical data $\mathbf{x} \in \mathbb{R}^d$ and produces a quantum state. The **feature map**

$$\Phi : \mathbb{R}^d \longrightarrow \mathcal{H}_n = (\mathbb{C}^2)^{\otimes n}, \qquad \mathbf{x} \mapsto |\phi(\mathbf{x})\rangle,$$

determines:

- what functions the circuit can represent (expressivity),
- the induced kernel on the data space,
- trainability (gradient norms depend on the map).

Principle

The encoding is part of the model. Choosing it well is as important as designing the variational ansatz. For PDEs, the encoding also determines which spatial frequencies the circuit can resolve.

Basis Encoding

Idea. Map a binary string $\mathbf{x} \in \{0,1\}^n$ to the computational basis state:

$$\mathbf{x} = (x_1, \ldots, x_n) \longmapsto |x_1 x_2 \cdots x_n\rangle.$$

Procedure.

1. Start in $|0\rangle^{\otimes n}$.
2. Apply X_i (NOT gate) on qubit i wherever $x_i = 1$.

Properties.

- Only n gates; depth $O(1)$.
- Requires n qubits for n bits.
- Superposition: encode N strings using $\lceil \log_2 N \rceil$ qubits *if* state preparation is efficient.
- Not suited to continuous features or PDEs.

Angle Encoding

Idea. Encode $x_i \in \mathbb{R}$ into the rotation angle of qubit i:

$$|\phi(\mathbf{x})\rangle = \bigotimes_{i=1}^{n} R_y(\pi x_i)|0\rangle = \bigotimes_{i=1}^{n} \begin{pmatrix} \cos(\pi x_i/2) \\ \sin(\pi x_i/2) \end{pmatrix}.$$

Rotation matrices for $k \in \{x, y, z\}$: $R_k(\alpha) = e^{-i\alpha\sigma_k/2}$,

$$R_y(\alpha) = \begin{pmatrix} \cos\frac{\alpha}{2} & -\sin\frac{\alpha}{2} \\ \sin\frac{\alpha}{2} & \cos\frac{\alpha}{2} \end{pmatrix}, \quad R_z(\alpha) = \begin{pmatrix} e^{-i\alpha/2} & 0 \\ 0 & e^{i\alpha/2} \end{pmatrix}.$$

- One feature per qubit; $n = d$. Depth $O(1)$.
- Product state — *no entanglement yet*; entanglement added by two-qubit gates in the variational layers.
- The output $f(x;\theta)$ is a *Fourier series* in x with frequencies determined by the generator spectrum.

Amplitude Encoding

Idea. Embed $\mathbf{x} \in \mathbb{R}^{2^n}$, $\|\mathbf{x}\| = 1$, into state amplitudes:

$$|\phi(\mathbf{x})\rangle = \sum_{j=0}^{2^n-1} x_j |j\rangle.$$

Capacity.

- n qubits encode 2^n features — *exponential compression*.
- Exploits the full Hilbert space.

Challenges.

- General state preparation requires $O(2^n)$ gates.
- Normalisation: $\sum_j x_j^2 = 1$.
- N data points need N circuit runs.

Read-out bottleneck

The exponential compression does *not* automatically give a speedup: reading out all 2^n amplitudes requires exponentially many measurements. This is the fundamental quantum read-out bottleneck relevant to all QPINNs.

Structured Feature Maps: IQP and ZZ

IQP feature map (Havlíček et al., Nature 2019):

$$|\phi(\mathbf{x})\rangle = U_\Phi(\mathbf{x})|+\rangle^{\otimes n}, \quad U_\Phi(\mathbf{x}) = e^{i\phi(\mathbf{x})}, \quad \phi(\mathbf{x}) = \sum_S c_S(\mathbf{x}) \bigotimes_{i\in S} Z_i.$$

Second-order ZZ feature map (two layers):

$$\begin{aligned} U_\Phi^{(2)}(\mathbf{x}) &= V_2(\mathbf{x}) H^{\otimes n} V_1(\mathbf{x}) H^{\otimes n}, \\ V_k(\mathbf{x}) &= \exp\Big(i \sum_i x_i Z_i + i \sum_{i<j} (\pi - x_i)(\pi - x_j) Z_i Z_j \Big). \end{aligned}$$

These encode two-body correlations; the resulting kernel $K(\mathbf{x}, \mathbf{x}') = \left|\langle\phi(\mathbf{x})|\phi(\mathbf{x}')\rangle\right|^2$ is believed to be classically hard to simulate. For QPINNs, IQP encoding introduces cross-frequency terms $\omega_i\omega_j$ that enrich the Fourier spectrum of the solution approximation.

Data Re-Uploading: Expressivity via Repetition

Key idea (Pérez-Salinas et al. 2020): encode $\mathbf{x}$ *at every layer*:

$$U(\mathbf{x}, \boldsymbol{\Theta}) = \prod_{\ell=1}^{L} \Big[W(\boldsymbol{\Theta}^{(\ell)})\, S(\mathbf{x}) \Big], \qquad S(\mathbf{x}) = \bigotimes_i R_y(\pi x_i).$$

Why it helps.

- A single qubit with L re-uploads represents any function $f : \mathbb{R} \to \mathbb{R}$ as a Fourier series up to frequency L.
- For PDEs: maximum resolvable frequency is $\omega_{\max} = Ln/2$ with n qubits and L re-uploading layers.

Fourier viewpoint:

$$f(\mathbf{x}; \boldsymbol{\Theta}) = \sum_{\boldsymbol{\omega} \in \Omega} c_{\boldsymbol{\omega}}(\boldsymbol{\Theta})\, e^{i\boldsymbol{\omega} \cdot \mathbf{x}}.$$

Encoding comparison for PDEs

Encoding	**Qubits**	$\omega_{\max}$
Angle (1 layer)	n	$n/2$
Re-uploading	n	$Ln/2$
IQP / ZZ	n	$n^2/4$
Amplitude	n	$2^n/2$

Design rule

Variational Quantum Circuits: Setup

For a parameterised circuit $U(\boldsymbol{\theta}) = \prod_j G_j(\theta_j)$ with $G_j(\theta_j) = e^{-i\theta_j P_j/2}$, $P_j^2 = \mathbb{1}$:

$$f(\boldsymbol{\theta}) = \langle\psi_0|U^\dagger(\boldsymbol{\theta})\,\hat{B}\,U(\boldsymbol{\theta})|\psi_0\rangle.$$

Parameter-shift rule: derivation

Write $U(\theta_j) = A_j G_j(\theta_j) B_j$. Using the Euler decomposition $G_j(\theta_j) = \cos\frac{\theta_j}{2}\,\mathbb{1} - i\sin\frac{\theta_j}{2}\,P_j$ and $P_j^2 = \mathbb{1}$:

$$\begin{aligned}\frac{\partial f}{\partial\theta_j} &= \mathrm{Re}\big\langle\psi_0|B_j^\dagger\big(-\tfrac{i}{2}P_j G_j^\dagger(\theta_j)\big)A_j^\dagger\hat{B}\,A_j G_j(\theta_j)B_j|\psi_0\big\rangle \times 2\\ &= \frac{1}{2}\big[f(\theta_j + \tfrac{\pi}{2}) - f(\theta_j - \tfrac{\pi}{2})\big].\end{aligned}$$

Properties: exact (not approximate); 2 circuit evaluations per parameter; generalises to $r\big[f(\theta_j + \frac{\pi}{4r}) - f(\theta_j - \frac{\pi}{4r})\big]$ for generators with

Second derivative:

$$\frac{\partial^2 f}{\partial\theta_j^2} = f(\theta_j{+}\pi) - 2f(\theta_j) + f(\theta_j{-}\pi),$$

Quantum Fisher Information Matrix

Definition (pure state $|\psi(\boldsymbol{\theta})\rangle$):

$$\boxed{F_{jk}(\boldsymbol{\theta}) = 4\,\mathrm{Re}(\langle\partial_j\psi|\partial_k\psi\rangle - \langle\partial_j\psi|\psi\rangle\langle\psi|\partial_k\psi\rangle)}$$

where $|\partial_j\psi\rangle = \partial|\psi(\boldsymbol{\theta})\rangle/\partial\theta_j$.

Geometric meaning. $\mathbf{F}$ is the **Fubini–Study metric** on the variational manifold:

$$ds^2 = \sum_{jk} F_{jk}\, d\theta_j\, d\theta_k.$$

It measures the *information-geometric distance* between neighbouring quantum states.

Covariance form. For product-of-rotations circuits:

$$F_{jk} = 4\,\mathrm{Cov}_\psi(G_j, G_k).$$

Circuit-based formula

Using two-fold parameter shifts on the state overlap $\mathcal{O}(\boldsymbol{\theta}) = |\langle\psi_{\mathrm{ref}}|\psi(\boldsymbol{\theta})\rangle|^2$:

$$F_{jk} = \tfrac{1}{2}\Big[\mathcal{O}_{j+,k+} - \mathcal{O}_{j+,k-} - \mathcal{O}_{j-,k+} + \mathcal{O}_{j-,k-}\Big]$$

where $\mathcal{O}_{j\pm,k\pm}$ denotes $\mathcal{O}$ at $(\theta_j \pm \frac{\pi}{2}, \theta_k \pm \frac{\pi}{2})$. Cost: $O(p^2)$ circuit evaluations.

Natural Gradient and the TDVP

Natural gradient (Amari 1998; Stokes et al. 2020):

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\mathbf{F}(\boldsymbol{\theta})^{+}\,\nabla_\theta C.$$

Derivation. Minimise $C(\boldsymbol{\theta}+\delta\boldsymbol{\theta})$ subject to fixed Bures distance: $\delta\boldsymbol{\theta}^\top \mathbf{F}\,\delta\boldsymbol{\theta} = \varepsilon^2$, giving $\delta\boldsymbol{\theta} \propto -\mathbf{F}^{-1}\nabla C$.
The update is *reparameterisation-invariant*: it always descends the steepest direction on $\mathcal{M}$.

TDVP equations of motion

From the Dirac–Frenkel principle, projecting $(i\partial_t - H)|\psi\rangle = 0$ onto the tangent space $\{|\partial_j\psi\rangle\}$:

$$\sum_k F_{jk}\,\dot\theta_k = 2\,\mathrm{Im}\langle\partial_j\psi|H|\psi\rangle$$

Imaginary-time TDVP = natural gradient VQE

Substituting $t \to -i\tau$:

$$\dot{\boldsymbol{\theta}} = -\mathbf{F}^{-1}\nabla_\theta E(\boldsymbol{\theta}).$$

Effective Dimension and Expressivity

Abbas et al. (2021) define the **effective dimension** via the average log-volume of the QFI:

$$d_{\text{eff}} = 2\,\frac{1}{\log N}\log\left[\frac{1}{V}\int_{\boldsymbol{\theta}}\sqrt{\det\left(I + \frac{N}{2\pi\ln N}\,\mathbf{F}(\boldsymbol{\theta})\right)}\,d\boldsymbol{\theta}\right],$$

where N is the number of training samples.

- High $d_{\text{eff}} \Rightarrow$ model fits many functions; good generalisation capacity.
- Some QNNs achieve higher d_{eff} than classical networks of comparable parameter count.
- $\mathbf{F} \approx 0$ (rank-deficient) $\Rightarrow$ parameters are redundant; barren-plateau-like flat directions.

QFI as trainability indicator

$$\left|\frac{\partial C}{\partial \theta_j}\right|^2 \leq \lambda_{\max}(\mathbf{F}) \cdot \mathrm{Var}_{\psi}(\hat{B})$$

(Cauchy–Schwarz on the Fubini–Study metric).

Small $\lambda_{\max}(\mathbf{F}) \Leftrightarrow$ all gradients are small $\Leftrightarrow$ barren plateau.

Barren Plateaus: Definition and Origin

Definition (McClean et al. 2018)

A circuit family $\{U(\boldsymbol{\theta})\}$ exhibits a **barren plateau** if, for any partial derivative:

$$\mathbb{E}_{\boldsymbol{\theta}}\left[\frac{\partial C}{\partial \theta_j}\right] = 0, \qquad \mathrm{Var}_{\boldsymbol{\theta}}\left[\frac{\partial C}{\partial \theta_j}\right] = O\left(\frac{1}{b^n}\right), \; b > 1.$$

Mathematical origin. For Haar-random U on $\mathrm{SU}(2^n)$:

$$\mathbb{E}_U\left[\langle \hat{O} \rangle\right] = \frac{\mathrm{Tr}\,\hat{O}}{2^n}, \qquad \mathrm{Var}\left[\frac{\partial C}{\partial \theta_j}\right] \sim \frac{\mathrm{Tr}\,\hat{O}^2}{4^n}.$$

For a local observable $\hat{O} = Z_k$: $\mathrm{Var} \sim 1/2^n$.

Concentration of measure (Lévy's lemma): in high dimensions, all Lipschitz functions concentrate near their mean with probability $\geq 1 - 2e^{-\Omega(2^{2n}\varepsilon^2)}$.

Consequence

Deep random circuits make $C(\boldsymbol{\theta}) \approx \mathrm{Tr}\,\hat{O}/2^n$ almost everywhere: landscape is flat. This is *not* a

Noise-Induced Barren Plateaus and Mitigation

Hardware noise (Wang et al. 2021): depolarising noise with rate ϵ per gate gives:

$$|\langle \hat{O} \rangle_{\text{noisy}}| \leq (1-\epsilon)^{pd} \, |\langle \hat{O} \rangle_{\text{ideal}}|,$$

$$\mathrm{Var}\left[\frac{\partial C_{\text{noisy}}}{\partial \theta_j}\right] \sim (1-\epsilon)^{2pd} \cdot \mathrm{Var}\left[\frac{\partial C_{\text{ideal}}}{\partial \theta_j}\right].$$

Affects even *shallow* circuits with many gates.

Mitigation strategies

Strategy	Mechanism
Local cost	poly(n) scaling
Problem ansatz	exploit symmetries
Layerwise training	$O(1)$ depth locally
Identity init	$O(1)$ gradients at start
Natural gradient	QFI-respecting steps
Shallow circuits	avoid Haar regime

QPINNs and BPs

Physics-inspired shallow ansätze (Green's function structure, Fourier modes) naturally avoid the random circuit regime — the physical structure of the PDE

Entanglement, QNTK, and Training Regimes

Entanglement entropy:
$S(\rho_A) = -\operatorname{Tr}(\rho_A \log \rho_A)$ governs the training regime:

Regime	$S(\rho_A)$	Training
Low	≈ 0	Classically simulable
Volume-law	$\Theta(n)$	Barren plateaus
Area-law	$O(\lvert\partial A\rvert)$	Trainable

Quantum Neural Tangent Kernel (QNTK):

$$\Theta(\mathbf{x}, \mathbf{x}') = \sum_j \frac{\partial f(\mathbf{x}; \boldsymbol{\theta})}{\partial \theta_j} \frac{\partial f(\mathbf{x}'; \boldsymbol{\theta})}{\partial \theta_j}.$$

- Governs training dynamics in the linearised regime.
- At a barren plateau: $\Theta \approx 0$ for all $\mathbf{x}, \mathbf{x}'$.
- Bound: $\Theta(\mathbf{x}, \mathbf{x}) \leq \lambda_{\max}(\mathbf{F})$.
- Connects QNN training to quantum kernel methods (QSVMs).

Classical PINNs: Raissi, Perdikaris, Karniadakis (2019)

A classical PINN approximates the solution of $\mathcal{N}[u](x,t) = 0$ on Ω by a neural network $u_\theta(x,t)$ minimising:

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N_r}\sum_{k=1}^{N_r}\left|\mathcal{N}[u_\theta](x_k,t_k)\right|^2}_{\text{PDE residual } \mathcal{L}_r} + \lambda \underbrace{\frac{1}{N_b}\sum_{k=1}^{N_b}\left|u_\theta(x_k^b,t_k^b) - g_k\right|^2}_{\text{BC/IC loss}}$$

Collocation points $\{x_k\}$ sampled in Ω; $\{x_k^b\}$ on $\partial\Omega$.

Automatic differentiation

Classical PINNs evaluate $\mathcal{N}[u_\theta]$ via autograd (no finite differences).

Spectral bias

Classical NNs learn low-frequency components much faster than high-frequency ones. This is the chief motivation for QPINNs with explicit, controllable frequency spectra.

Examples

Poisson $-\nabla^2 u = f$; Heat $\partial_t u = \kappa\nabla^2 u$;

PINN Loss Landscape and the Neural Tangent Kernel

Residual at collocation point x_k:
$r_k(\theta) = \mathcal{D}[f_\theta](x_k)$.

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_k r_k^2, \quad \nabla_\theta \mathcal{L} = \frac{2}{N}\sum_k r_k \nabla_\theta r_k.$$

The **Neural Tangent Kernel (NTK)** governs training dynamics:

$$\Theta_{kl} = \nabla_\theta f_\theta(x_k)^\top \nabla_\theta f_\theta(x_l).$$

Eigenvalues of Θ determine convergence rates per frequency component.

Failure modes

1. **Spectral bias:** high-frequency modes converge slowly
2. **Loss balance:** $\mathcal{L}_r$ and $\mathcal{L}_b$ compete
3. **Stiff equations:** chaotic landscape
4. **High dimensionality:** exponential cost

Quantum motivation

QPINNs address points 1 and 4: richer frequency spectra via entanglement; quantum memory scales as $O(2^n)$ with n qubits.

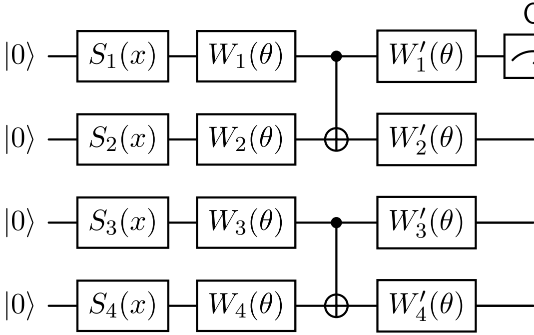
Replace the classical network $u_\theta(x)$ with a variational quantum circuit output:

$$u_\theta(x) = \langle 0|U^\dagger(x,\theta)\,O\,U(x,\theta)|0\rangle$$

where:

- $U(x,\theta) = W_L(\theta)\cdots W_1(\theta)\,S(x)$
- $S(x) = \bigotimes_i R_y(\pi x)$: data encoding
- $W_l(\theta)$: variational Pauli layers
- $O = Z_0$: fixed observable giving scalar output

The output $u_\theta(x) \in [-1,1]$ approximates the normalised PDE solution.



$S_i(x)$: input encoding; W_i: variational layers; $O = Z_0$.

The QPINN Loss Function

For $\mathcal{D}[u](x) = f(x)$ on Ω with $u = g$ on $\partial\Omega$:

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N_r}\sum_k \left|\mathcal{D}[u_\theta](x_k) - f(x_k)\right|^2}_{\mathcal{L}_r} + \lambda \underbrace{\frac{1}{N_b}\sum_k \left|u_\theta(x_k^b) - g(x_k^b)\right|^2}_{\mathcal{L}_b}.$$

Gradient w.r.t. θ_μ:

$$\frac{\partial \mathcal{L}_r}{\partial \theta_\mu} = \frac{2}{N_r}\sum_k r_k \frac{\partial \mathcal{D}[u_\theta](x_k)}{\partial \theta_\mu}.$$

For a *linear* operator $\mathcal{D}$:

$$\frac{\partial \mathcal{D}[u_\theta]}{\partial \theta_\mu} = \mathcal{D}\left[\frac{\partial u_\theta}{\partial \theta_\mu}\right],$$

so $\mathcal{D}$ and the parameter-shift commute.

Two shift rules working together

- **Input-shift**: computes spatial derivatives $\mathcal{D}[u_\theta]$
- **Parameter-shift**: computes gradients $\partial u_\theta / \partial \theta_\mu$

Total cost per gradient step: $O(N_p \times N_r)$ circuit evaluations.

The Input-Shift Rule: Spatial Derivatives

For angle encoding $S(x) = e^{-ixP/2}$ (Pauli P, eigenvalues $\pm\frac{1}{2}$):

First derivative (input-shift rule):

$$\frac{\partial u_\theta}{\partial x} = \frac{1}{2}\left[u_\theta(x+\tfrac{\pi}{2}) - u_\theta(x-\tfrac{\pi}{2})\right]$$

Second derivative:

$$\frac{\partial^2 u_\theta}{\partial x^2} = u_\theta(x+\tfrac{\pi}{2}) - 2u_\theta(x) + u_\theta(x-\tfrac{\pi}{2})$$

n-th order:

$$\partial_x^n u_\theta = \sum_{k=0}^{n} (-1)^k \binom{n}{k} u_\theta\left(x + \frac{(n-2k)\pi}{4}\right)$$

Key result

All spatial derivatives of $u_\theta(x)$ are computable by evaluating the *same circuit* at shifted input values. No ancilla qubits; no additional circuit depth.

Cost budget

k-th order derivative at one point: $k+1$ circuit runs.

Laplacian ∂_{xx}: **3 runs** at $x-\frac{\pi}{2}$, x, $x+\frac{\pi}{2}$.

Unified Shift Framework: One Rule, Two Applications

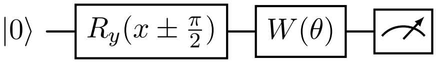
For *any* gate $e^{-i\phi P/2}$ (Pauli P):

$$\frac{\partial}{\partial \phi}\langle O\rangle_\phi = \frac{1}{2}\big[\langle O\rangle_{\phi+\pi/2} - \langle O\rangle_{\phi-\pi/2}\big].$$

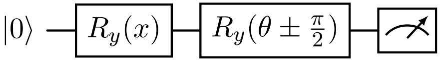
Two interpretations:

- $\phi = \theta_\mu$ (variational parameter): **parameter-shift** — gradient for training
- $\phi = x$ (input encoding): **input-shift** — spatial derivative of PDE solution

They are *the same formula*, applied to different gates in the circuit.



Input-shift: evaluates $\partial_x u_\theta$.



Parameter-shift: evaluates $\partial_\theta u_\theta$.

Commutativity (linear PDEs)

For linear $\mathcal{D}$, the parameter-shift and input-shift commute:

$$\frac{\partial}{\partial \theta_\mu}\mathcal{D}[u_\theta](x) = \mathcal{D}\left[\frac{\partial u_\theta}{\partial \theta_\mu}\right](x).$$

Multi-Dimensional PDEs and the Variational Form

Multi-dimensional encoding. For $u(x_1, \ldots, x_d)$ with n_i qubits for x_i:

$$S(\mathbf{x}) = R_y(\pi x_1)^{\otimes n_1} \otimes \cdots \otimes R_y(\pi x_d)^{\otimes n_d}.$$

Cross-derivatives via simultaneous shifts:

$$\partial_{x_i}\partial_{x_j} u_\theta = \text{shift in } x_i \text{ and } x_j.$$

Total qubits: $n = n_1 + \cdots + n_d$. Classical PINN: $O(\varepsilon^{-d})$ parameters; QPINN: $O(d)$ qubits if solution has tensor-product structure.

Variational (Ritz) form

For Poisson $-\nabla^2 u = f$, minimise the energy:

$$\mathcal{L}_{\text{Ritz}}(\theta) = \sum_k w_k \left[\frac{1}{2} \left(\frac{du_\theta}{dx}\bigg|_{x_k} \right)^2 - f(x_k)\, u_\theta$$

Needs only first derivatives; smoother loss landscape.

Ritz-QPINN = quantum Galerkin method

The quantum ansatz $u_\theta(x)$ plays the role of the finite element basis, but spans an

The 1D Poisson Equation

Problem:

$$-\frac{d^2u}{dx^2} = f(x), \quad x \in (0,1), \quad u(0) = u(1) = 0.$$

Exact solution for $f(x) = \pi^2 \sin(\pi x)$:

$$u^\star(x) = \sin(\pi x).$$

QPINN residual loss:

$$\mathcal{L}_r = \frac{1}{N_r} \sum_{k=1}^{N_r} \left[-\frac{d^2 u_\theta}{dx^2}\bigg|_{x_k} - f(x_k) \right]^2 .$$

Boundary loss: $\mathcal{L}_b = u_\theta(0)^2 + u_\theta(1)^2$.

Second derivative via input-shift:

$$\frac{d^2 u_\theta}{dx^2}\bigg|_x = u_\theta(x+\tfrac{\pi}{2}) - 2u_\theta(x) + u_\theta(x-\tfrac{\pi}{2}).$$

This uses $S(x) = e^{-i\pi x P/2}$ with $P = Y \otimes \cdots$: exactly 3 circuit evaluations per collocation point.

Circuit count per gradient step

N_r interior points $\times 3$ (Laplacian) $\times 2N_p$ (parameter shift) $= 6N_pN_r$ total. For $N_p = 8$, $N_r = 20$: 960 evaluations per step.

Python: Circuit and Gate Definitions

```python
import numpy as np

def ry(t):
    c, s = np.cos(t / 2), np.sin(t / 2)
    return np.array([[c, -s], [s, c]], dtype=complex)

I2   = np.eye(2, dtype=complex)
CNOT = np.array([[1,0,0,0],[0,1,0,0],[0,0,0,1],[0,0,1,0]], dtype=complex)
Z0   = np.kron(np.diag([1., -1.]).astype(complex), I2)  # Z on qubit 0

def var_layer(p):
    """Ry(p0)xRy(p1)␣->␣CNOT␣->␣Ry(p2)xRy(p3)"""
    return CNOT @ np.kron(ry(p[0]), ry(p[1]))

def circuit(x, params):
    """
␣␣␣␣2-qubit␣QPINN␣circuit␣for␣1D␣PDE.
␣␣␣␣Encoding:␣␣Ry(pi*x)␣on␣each␣qubit.
␣␣␣␣Ansatz:␣␣␣␣2␣variational␣layers␣(8␣parameters).
␣␣␣␣Output:␣␣␣␣<Z_0>␣in␣[-1,␣1].
␣␣␣␣"""
    state = np.zeros(4, dtype=complex); state[0] = 1.0
```

Python: Input-Shift Derivatives and Loss

```python
SHIFT = np.pi / 2   # shift for Ry generator (eigenvalues +/- 1/2)

def du_dx(x, params):
    """First derivative via input-shift rule."""
    return 0.5 * (circuit(x + SHIFT, params)
                - circuit(x - SHIFT, params))

def d2u_dx2(x, params):
    """Second derivative (Laplacian at one point = 3 circuit runs)."""
    return (circuit(x + SHIFT, params)
          - 2 * circuit(x, params)
          + circuit(x - SHIFT, params))

# Collocation and boundary points
x_r      = np.linspace(0.05, 0.95, 20)      # interior collocation
x_b      = np.array([0.0, 1.0])             # Dirichlet boundary
lambda_bc = 10.0

def loss(params):
    residuals = np.array([
        -d2u_dx2(x, params) - np.pi**2 * np.sin(np.pi * x) for x in x_r
    ])
```

Python: Training Loop (Adam Optimiser)

```
np.random.seed(7)
params = np.random.uniform(0, np.pi, 8)    # 2 layers x 4 params

# Adam hyperparameters
lr, b1, b2, eps = 0.15, 0.9, 0.999, 1e-8
m, v = np.zeros(8), np.zeros(8)
history = []

for step in range(300):
    g = grad_loss(params)
    t = step + 1
    m  = b1 * m + (1 - b1) * g
    v  = b2 * v + (1 - b2) * g**2
    mc = m / (1 - b1**t);  vc = v / (1 - b2**t)
    params -= lr * mc / (np.sqrt(vc) + eps)
    history.append(loss(params))
    if (step + 1) % 75 == 0:
        print(f"Step {step+1:4d}  loss = {history[-1]:.6f}")

# Evaluate and compare
x_test  = np.linspace(0, 1, 100)
u_qpinn = np.array([circuit(x, params) for x in x_test])
```

Results and Interpretation

Expected output:

- L2 error $\lesssim 0.05$ after 300 steps
- Loss decreases by 2–3 orders of magnitude
- QPINN traces $\sin(\pi x)$ accurately

Improving accuracy:

- More qubits (wider frequency spectrum)
- More variational layers (richer coefficients)
- More collocation points N_r
- Longer training / lower learning rate
- Natural gradient (QFI) instead of Adam

Connection to Section 2

The Adam optimiser treats all parameters equally (Euclidean geometry). Replacing it with $\delta\theta = -\eta\,\mathbf{F}^{-1}\nabla_\theta\mathcal{L}$ (natural gradient) respects the Fubini–Study metric and typically converges faster — particularly important for stiff PDEs where the loss landscape is highly anisotropic.

Barren plateau warning

Adding more layers improves expressivity but risks entering the barren plateau regime. Physics-inspired ansätze mitigate

Expressivity of QPINNs: Fourier Analysis

Theorem (Schuld et al. 2021). The output of any QNN with data-encoding gates $e^{-ixP/2}$ is a *partial Fourier series*:

$$u_\theta(x) = \sum_{\omega \in \Omega} c_\omega(\theta)\, e^{i\omega x}$$

where:

$$\Omega \subseteq \left\{\omega_1 + \cdots + \omega_n : \omega_i \in \{0, \pm\tfrac{1}{2}\}\right\}.$$

Frequencies Ω fixed by encoding; coefficients c_ω tuned by θ.

For PDEs: highest frequency in Ω limits the shortest spatial scale resolved.

Error decomposition

$$\|u_{\theta^\star} - u^\star\|_{L^2} \leq \underbrace{\varepsilon_{\text{approx}}}_{\text{expressivity}} + \underbrace{\varepsilon_{\text{opt}}}_{\text{optimisation}} + \underbrace{\varepsilon_{\text{stat}}}_{\text{samplir}}$$

$\varepsilon_{\text{approx}} \sim \sum_{\omega \notin \Omega} |\hat{u}^\star_\omega|$; $\varepsilon_{\text{stat}} \sim N_r^{-1/2}$.

Circuit sizing

For target frequency $\omega^\star$, choose n qubits and L re-uploading layers such that $Ln \geq 2\omega^\star$.

Extension: Time-Dependent PDEs

For $u(x,t)$, encode space and time jointly:

$$S(x,t) = R_y(\pi x)^{\otimes n_x} \otimes R_y(\pi t)^{\otimes n_t}.$$

1D heat equation: $\partial_t u - \kappa \partial_{xx} u = 0$
Input shifts for PDE residual:

$$\partial_t u \approx \tfrac{1}{2}[u(x, t+\tfrac{\pi}{2}) - u(x, t-\tfrac{\pi}{2})]$$
$$\partial_{xx} u \approx u(x+\tfrac{\pi}{2}, t) - 2u(x,t) + u(x-\tfrac{\pi}{2}, t)$$

Each residual point: 5 circuit evaluations.
Add IC loss:
$\mathcal{L}_{\mathrm{IC}} = N_x^{-1} \sum_k |u_\theta(x_k, 0) - u_0(x_k)|^2$.

Dimensional scaling

d-dimensional PDE: $n_x + n_t = n$ total qubits. Classical PINN cost: $O(\varepsilon^{-d})$. QPINN cost: $O(d)$ qubits if solution has tensor-product structure. Advantage largest when $d \gg 1$.

Exact: $u(x,t) = e^{-\pi^2 t} \sin(\pi x)$

Compare QPINN to exact solution; measure L^∞ error across the space-time domain.

The Schrödinger Equation as a QPINN

Time-dependent Schrödinger equation:

$$i\hbar\partial_t\psi = H\psi, \quad H = -\frac{\hbar^2}{2m}\partial_{xx} + V(x).$$

Split $\psi = \psi_R + i\psi_I$:

$$\begin{aligned}\partial_t\psi_R &= \quad \tfrac{\hbar}{2m}\partial_{xx}\psi_I - V\psi_I/\hbar \\ \partial_t\psi_I &= -\tfrac{\hbar}{2m}\partial_{xx}\psi_R + V\psi_R/\hbar\end{aligned}$$

Two QPINNs: $\psi_R \approx \langle Z_0\rangle_{x,t,\theta_R}$ and $\psi_I \approx \langle Z_0\rangle_{x,t,\theta_I}$. All derivatives by input-shift; $V(x)$ analytically.

Bridge to Hamiltonian simulation

The QPINN Schrödinger formulation is the *classical simulation* of quantum time evolution. On a quantum device, Hamiltonian simulation provides $e^{-iHt/\hbar}$ exactly — the QPINN bridges continuous PDE solving and digital quantum simulation.

Time-independent: VQE connection

The eigenvalue problem $H\psi = E\psi$ is solved by minimising the Rayleigh quotient:

$\langle\psi_\theta|H|\psi_\theta\rangle$

Barren Plateaus in QPINNs: Additional Sources

QPINNs inherit the general barren plateau problem (Section 4) and have additional sources specific to PDE training:

General BP (random deep circuits):

$$\mathrm{Var}\left(\frac{\partial \mathcal{L}}{\partial \theta_\mu}\right) \sim \frac{1}{2^n}.$$

QPINN-specific sources:

- Residual $r_k \approx 0$ early in training: gradient is small even without BPs
- Loss imbalance: $\mathcal{L}_r \gg \mathcal{L}_b$ makes BC enforcement slow
- Stiff PDEs: widely varying eigenvalues of the differential operator

QPINN-specific mitigation

1. **Physics-inspired ansatz:** layer structure mirrors the Green's function
2. **Incremental frequency training:** start with low modes, add higher ones
3. **Adaptive collocation:** concentrate points where residual is large
4. **Loss weighting:** normalise $\mathcal{L}_r$ and $\mathcal{L}_b$ by their gradient norms
5. **Natural gradient (QFI):**

QPINN vs. Classical PINN: Summary

	Classical PINN	QPINN
Ansatz	Deep MLP $u_\theta(x)$	Circuit $\langle O \rangle_{x,\theta}$
Spatial deriv.	Automatic differentiation	Input-shift rule (exact)
Parameter grad.	Backpropagation	Parameter-shift rule (exact)
Frequency	Arbitrary (spectral bias)	Bounded, explicit Ω
Memory	$O(N_p)$ params	$O(n)$ params, 2^n amplitudes
High-d PDE	Exponential cost	Potential $O(d)$ qubit advantage
Optimiser geom.	Euclidean	QFI / Fubini–Study (natural grad.)
Advantage	Production-ready	High-d, quantum PDEs

Key Equations

Concept	Formula
Parameter-shift rule	$\frac{\partial f}{\partial \theta_j} = \frac{1}{2}[f(\theta_j + \frac{\pi}{2}) - f(\theta_j - \frac{\pi}{2})]$
Input-shift rule	$\frac{\partial u_\theta}{\partial x} = \frac{1}{2}[u_\theta(x + \frac{\pi}{2}) - u_\theta(x - \frac{\pi}{2})]$
Laplacian (2nd order)	$\partial_{xx} u_\theta = u_\theta(x + \frac{\pi}{2}) - 2u_\theta(x) + u_\theta(x - \frac{\pi}{2})$
QFI (pure state)	$F_{jk} = 4\,\mathrm{Re}[\langle \partial_j \psi \vert \partial_k \psi \rangle - \langle \partial_j \psi \vert \psi \rangle \langle \psi \vert \partial_k \psi \rangle]$
Natural gradient	$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\mathbf{F}^{-1} \nabla C$
TDVP	$\sum_k F_{jk} \dot{\theta}_k = 2\,\mathrm{Im}\langle \partial_j \psi \vert H \vert \psi \rangle$
Barren plateau	$\mathrm{Var}[\partial C / \partial \theta_j] \sim 1/2^n$
QPINN loss	$\mathcal{L} = \Vert \mathcal{D}[u_\theta] - f \Vert^2 + \lambda \Vert u_\theta - g \Vert^2_{\partial\Omega}$

Exercises

1. **Parameter-shift by hand.** For $f(\theta) = \langle 0|R_y(\theta)^\dagger Z R_y(\theta)|0\rangle$, apply the parameter-shift rule and verify by direct differentiation.
2. **QFI of a single qubit.** For $|\psi(\theta)\rangle = R_y(\theta)|0\rangle$, compute F_{11} via the Fubini–Study formula and interpret geometrically.
3. **Barren plateau scaling.** Numerically estimate $\mathrm{Var}[\partial C/\partial\theta_j]$ for $n = 2, \ldots, 8$ qubit random circuits of depth $d = 2n$. Verify the $O(1/2^n)$ scaling.
4. **Harmonic oscillator QPINN.** Modify the Poisson code to solve $u'' + \omega^2 u = 0$, $u(0) = 0$, $u'(0) = 1$. Compare to $u(x) = \sin(\omega x)/\omega$.
5. **Second-order input-shift identity.** Prove that for $S(x) = e^{-ixP/2}$ with eigenvalues $\pm\frac{1}{2}$:

$$\frac{d^2 u_\theta}{dx^2} = u_\theta(x+\tfrac{\pi}{2}) - 2u_\theta(x) + u_\theta(x-\tfrac{\pi}{2}).$$

6. **Natural gradient QPINN.** Replace Adam with the diagonal QFI natural gradient in the Poisson solver and compare convergence.

References

QNN foundations

1. J.R. McClean et al., *Barren plateaus*, Nat. Commun. **9**, 4812 (2018).
2. M. Cerezo et al., *Cost-function-dependent BPs*, Nat. Commun. **12**, 1791 (2021).
3. J. Stokes et al., *Quantum natural gradient*, Quantum **4**, 269 (2020).
4. A. Pérez-Salinas et al., *Data re-uploading*, Quantum **4**, 226 (2020).
5. A. Abbas et al., *Power of QNNs*, Nat. Comput. Sci. **1**, 403 (2021).
6. A. Havlíček et al., *Supervised learning with quantum-enhanced feature spaces*, Nature **567**, 209 (2019).

Classical PINNs

9. M. Raissi et al., *Physics-informed NNs*, J. Comput. Phys. **378**, 686 (2019).
10. S. Wang et al., *When and why PINNs fail*, J. Comput. Phys. **449**, 110768 (2022).

Quantum PINNs and expressivity

11. A. Kyriienko et al., *Solving nonlinear DEs with differentiable quantum circuits*, PRA **103**, 052416 (2021).
12. M. Schuld et al., *Effect of data encoding on expressive power*, PRA **103**, 032430 (2021).
13. M. Cerezo et al., *Variational quantum*